

linear__environment

October 9, 2025

1 Linear Economy Estimation (SVAR)

Please note that this notebook uses a venv which points to a base python version of **3.13.7**, some functionality may be limited if using an older version of python

1.1 All Imports

Libraries used:

- Vanilla Modules:
 - typing (optional)
 - * just for static type checking and code hygiene, not functionally required.
 - dataclasses (optional)
 - * class type of choice for SVAR return object, also not necessary but functionally optimal.
 - datetime
 - * Used for datetime.now() to set the end date bound for out of sample data.
- 3rd party:
 - fedfred
 - * This is the library which I wrote, published, and currently maintain under the MIT License
 - * Makes API requests to the FRED database and returns strongly typed and structured objects as well as dataframes to reduce boilerplate code.
 - pandas
 - * dataframe backend of choice, if computational speed or parallelization becomes a necessity the fedfred backend accomodates dask and polars.
 - numpy
 - * for mathematical ops and pandas interop
 - statsmodels
 - * for OLS and potentially VAR model if necessary.
 - matplotlib
 - * Used for plotting results.

```
[29]: %pip install fedfred pandas numpy statsmodels matplotlib ipympl ipywidgets  
      ↪ jupyterlab-lsp nbdtm --quiet
```

Note: you may need to restart the kernel to use updated packages.

```
[30]: import fedfred as fed
import pandas as pd
import numpy as np
from typing import Union, Tuple
from dataclasses import dataclass
import statsmodels.api as sm
from statsmodels.stats.stattools import durbin_watson
from statsmodels.stats.diagnostic import acorr_breusch_godfrey
import datetime
import matplotlib.pyplot as plt
```

```
[31]: %matplotlib widget
```

1.2 Model Assumptions and Static Inputs

According to the paper static inputs are as follows:

- training data is bound from 1987 Q3 to 2007 Q2
 - we can update these values for recency, if we want to be predictive, or leave them if we want to compare recent interest rate decisions with predictive outputs
- all data used is quarterly as specified by the paper.

```
[32]: # Training data Time Period: Q3 1987 - Q2 2007
start_date = "1987-07-01"
end_date = "2007-06-30"

# Training data Observation Frequency
freq = "q" # Quarterly

# Fred client
import fedfred as fed
fred = fed.FredAPI(api_key="7ab121fb17773e187bb6508e83e411da", cache_mode=True,
↪ cache_size=256)
```

1.3 Data Helpers

1.3.1 Pandas Dataframe to Series Conversion Function

Simple function for converting `pandas.DataFrames` of the standard return format for `fedfred.FredAPI.get_series_observations` to a `pandas.Series`

```
[33]: import pandas as pd
import numpy as np
from typing import Union

def to_series(x: Union[pd.Series, pd.DataFrame], name: str) -> pd.Series:
    """
    Accepts a Series or a DataFrame with 'date'/'value' columns (fedfred style)
    ↪ and returns a float Series with DatetimeIndex and the given `name`.
    """
```

Args:

x (pd.Series / pd.DataFrame): Input data to be converted.

name (str): Name to assign to the resulting Series.

Returns:

pd.Series: A float Series with DatetimeIndex and the given `name`.

Raises:

TypeError: If the input is neither a pd.Series nor a pd.DataFrame.

```
"""
if isinstance(x, pd.Series):
    s = x.copy()
    s.index = pd.to_datetime(s.index)
    s = s.astype(float)
    s.name = name
    return s

if not isinstance(x, pd.DataFrame):
    raise TypeError("Input must be a pd.Series or pd.DataFrame")

df = x.copy()

# 1) Identify the datetime index/column
if df.index.name and df.index.name.lower() == "date":
    idx = pd.to_datetime(df.index)
elif "date" in df.columns:
    idx = pd.to_datetime(df["date"])
else:
    # try any column with 'date' in the name
    cand = next((c for c in df.columns if "date" in c.lower()), None)
    if cand is not None:
        idx = pd.to_datetime(df[cand])
    else:
        # last resort: try to_datetime on the current index
        idx = pd.to_datetime(df.index)

# 2) Identify the value column (prefer 'value', else first numeric)
if "value" in df.columns:
    vals = pd.to_numeric(df["value"], errors="coerce")
else:
    num_cols = [c for c in df.columns if np.issubdtype(df[c].dtype, np.
↪number)]
    if not num_cols:
        # take the last column and coerce
        vals = pd.to_numeric(df.iloc[:, -1], errors="coerce")
    else:
```

```

        vals = pd.to_numeric(df[num_cols[0]], errors="coerce")

    s = pd.Series(vals.values, index=idx, name=name).astype(float)
    # Clean up: sort, dedupe, and ensure DatetimeIndex
    s = s[~s.index.duplicated(keep="last")].sort_index()
    s.index = pd.to_datetime(s.index)
    return s

```

1.4 Data Retrieval and Preparation

1.4.1 Data Fetching

- π_t -> Quarterly Inflationary Metric (YoY%)
 - For our inflationary measure we use the **GDP: Implicit Price Deflator** as specified in the paper.
 - FRED Series ID: **GDPDEF**
- Y_t -> Quarterly Real Output
 - For our real production output metric we use **Real GDP**
 - FRED Series ID: **GDPC1**
- Y_t^* -> Quarterly Potential Output
 - For our potential production output metric we use **Potential GDP**
 - FRED Series ID: **GDPPOT**
- i_t -> Quarterly Interest Rate
 - For our interest rate metric we use **Effective Federal Funds Rate**
 - FRED Series ID: **FEDFUNDS**

```

[34]: # Fetch data from FRED
gdpdef_yoy = fred.get_series_observations(
    series_id="GDPDEF",
    observation_start=start_date,
    observation_end=end_date,
    frequency=freq,
    units="pc1" # YoY percent change
)

real_gdp = fred.get_series_observations(
    series_id="GDPC1",
    observation_start=start_date,
    observation_end=end_date,
    frequency=freq,
    units="lin" # Linear units (level)
)

potential_gdp = fred.get_series_observations(
    series_id="GDPPOT",
    observation_start=start_date,
    observation_end=end_date,
    frequency=freq,

```

```

units="lin" # Linear units (level)
)

effective_ffr = fred.get_series_observations(
    series_id="FEDFUNDS",
    observation_start=start_date,
    observation_end=end_date,
    frequency=freq,
    aggregation_method="avg" # Aggregate daily data to quarterly by averaging
    ↪(specificly important for interest rates)
)

# Check data
print(gdpdef_yoy.head())
print(real_gdp.head())
print(potential_gdp.head())
print(effective_ffr.head())

```

	realtime_start	realtime_end	value
date			
1987-07-01	2025-10-03	2025-10-03	2.66122
1987-10-01	2025-10-03	2025-10-03	2.91876
1988-01-01	2025-10-03	2025-10-03	3.06577
1988-04-01	2025-10-03	2025-10-03	3.35274
1988-07-01	2025-10-03	2025-10-03	3.80207

	realtime_start	realtime_end	value
date			
1987-07-01	2025-10-03	2025-10-03	9162.024
1987-10-01	2025-10-03	2025-10-03	9319.332
1988-01-01	2025-10-03	2025-10-03	9367.502
1988-04-01	2025-10-03	2025-10-03	9490.594
1988-07-01	2025-10-03	2025-10-03	9546.206

	realtime_start	realtime_end	value
date			
1987-07-01	2025-10-03	2025-10-03	9197.770225
1987-10-01	2025-10-03	2025-10-03	9270.501462
1988-01-01	2025-10-03	2025-10-03	9343.141538
1988-04-01	2025-10-03	2025-10-03	9416.313106
1988-07-01	2025-10-03	2025-10-03	9489.692044

	realtime_start	realtime_end	value
date			
1987-07-01	2025-10-02	2025-10-02	6.84
1987-10-01	2025-10-02	2025-10-02	6.92
1988-01-01	2025-10-02	2025-10-02	6.66
1988-04-01	2025-10-02	2025-10-02	7.16
1988-07-01	2025-10-02	2025-10-02	7.98

1.4.2 Series Construction

```
[35]: # Convert to Series with proper names
pi = to_series(gdpdef_yoy, "pi")
y_real = to_series(real_gdp, "y_real")
y_pot = to_series(potential_gdp, "y_pot")
i = to_series(effective_ffr, "i")

# Check converted data
print(pi.head())
print(y_real.head())
print(y_pot.head())
print(i.head())
```

date

1987-07-01	2.66122
1987-10-01	2.91876
1988-01-01	3.06577
1988-04-01	3.35274
1988-07-01	3.80207

Name: pi, dtype: float64

date

1987-07-01	9162.024
1987-10-01	9319.332
1988-01-01	9367.502
1988-04-01	9490.594
1988-07-01	9546.206

Name: y_real, dtype: float64

date

1987-07-01	9197.770225
1987-10-01	9270.501462
1988-01-01	9343.141538
1988-04-01	9416.313106
1988-07-01	9489.692044

Name: y_pot, dtype: float64

date

1987-07-01	6.84
1987-10-01	6.92
1988-01-01	6.66
1988-04-01	7.16
1988-07-01	7.98

Name: i, dtype: float64

YoY Inflation in % using log-difference over 4 quarters: $\pi_t = 100 \times \left(\frac{P_t}{P_{t-4}} - 1 \right)$

We don't need to calculate this as FRED made the transformation for us when we specified the `units="pc1"` arg.

```
[36]: # Double check var assignment
print(pi.head())
print(pi.tail())
```

```
date
1987-07-01    2.66122
1987-10-01    2.91876
1988-01-01    3.06577
1988-04-01    3.35274
1988-07-01    3.80207
Name: pi, dtype: float64
date
2006-04-01    3.35642
2006-07-01    3.13805
2006-10-01    2.66316
2007-01-01    2.91019
2007-04-01    2.72883
Name: pi, dtype: float64
```

Output Gap Percent Deviation: $y_t = 100 \times \left(\frac{Y_t}{Y_t^*} - 1 \right)$

```
[37]: y_gap = 100.0 * (y_real / y_pot - 1.0)
y_gap.name = "y"

# Check calculated series
print(y_gap.head())
print(y_gap.tail())
```

```
date
1987-07-01   -0.388640
1987-10-01    0.526730
1988-01-01    0.260731
1988-04-01    0.788853
1988-07-01    0.595530
Name: y, dtype: float64
date
2006-04-01    1.319357
2006-07-01    0.978513
2006-10-01    1.380249
2007-01-01    1.210046
2007-04-01    1.336951
Name: y, dtype: float64
```

Policy rate: i_t

```
[38]: # Double check var assignment
print(i.head())
print(i.tail())
```

```

date
1987-07-01    6.84
1987-10-01    6.92
1988-01-01    6.66
1988-04-01    7.16
1988-07-01    7.98
Name: i, dtype: float64
date
2006-04-01    4.91
2006-07-01    5.25
2006-10-01    5.25
2007-01-01    5.26
2007-04-01    5.25
Name: i, dtype: float64

```

Merge and Check

```

[39]: # Merge
df = pd.concat([pi, y_gap, i], axis=1).dropna()
df.index = pd.PeriodIndex(df.index, freq="Q") # Use a quarterly PeriodIndex

# Check
print(df.head())
print(df.tail())
print(df.index.min(), "→", df.index.max())

# Additional Sanity Check:
assert df.index.min() == pd.Period("1987Q3", "Q")
assert df.index.max() == pd.Period("2007Q2", "Q")
assert len(df) == 80
assert not df.isna().any().any()

```

```

      pi      y      i
date
1987Q3  2.66122 -0.388640  6.84
1987Q4  2.91876  0.526730  6.92
1988Q1  3.06577  0.260731  6.66
1988Q2  3.35274  0.788853  7.16
1988Q3  3.80207  0.595530  7.98
      pi      y      i
date
2006Q2  3.35642  1.319357  4.91
2006Q3  3.13805  0.978513  5.25
2006Q4  2.66316  1.380249  5.25
2007Q1  2.91019  1.210046  5.26
2007Q2  2.72883  1.336951  5.25
1987Q3 → 2007Q2

```

1.5 Linear SVAR Environment Estimation

1.6 Fitting Helpers

1.6.1 SVAR Return Object

Return object for SVAR Function (OLS) is simply a python `dataclasses.dataclass` containing `statsmodels.api.regression.linear_model.RegressionResultsWrapper`, `pandas.DataFrame`, and `pandas.Series` as instance attributes.

```
[40]: from dataclasses import dataclass
import statsmodels.api as sm

@dataclass(frozen=True)
class SVARResults:
    """
    Container for the linear SVAR (recursive OLS) results.

    Attributes:
        mod_y (statsmodels.regression.linear_model.RegressionResultsWrapper)
        ↪ Fitted OLS model for the output-gap equation.
        mod_pi (statsmodels.regression.linear_model.RegressionResultsWrapper)
        ↪ Fitted OLS model for the inflation equation.
        X_y (pd.DataFrame) Design matrix used in the output-gap regression.
        X_pi (pd.DataFrame) Design matrix used in the inflation regression.
        resid_y (pd.Series) Residuals (innovations) from the output-gap
        ↪ regression.
        resid_pi (pd.Series) Residuals (innovations) from the inflation
        ↪ regression.
    """
    mod_y: sm.regression.linear_model.RegressionResultsWrapper
    mod_pi: sm.regression.linear_model.RegressionResultsWrapper
    X_y: pd.DataFrame
    X_pi: pd.DataFrame
    resid_y: pd.Series
    resid_pi: pd.Series
```

1.6.2 SVAR Lag Dropping Function

As per the paper after performing recursive SVAR(2) via OLS we drop all lags that are insignificant ($p > \alpha$ for $\alpha = 0.10$)

```
[41]: from typing import Tuple

def drop_insignificant_lag2(
    model: sm.regression.linear_model.RegressionResultsWrapper,
    X: pd.DataFrame,
    y: pd.Series,
    alpha: float = 0.10,
```

```

) -> Tuple[sm.regression.linear_model.RegressionResultsWrapper, pd.DataFrame]:
    """
    Drop any lag-2 regressors with p-values > alpha, then refit OLS.

    Args:
        model (sm.regression.linear_model.RegressionResultsWrapper): Fitted OLS
        ↪ model.
        X (pd.DataFrame): Design matrix used in the regression.
        y (pd.Series): Target variable used in the regression.
        alpha (float, optional): Significance level for dropping lag-2 terms.
        ↪ Defaults to 0.10.

    Returns:
        Tuple[sm.regression.linear_model.RegressionResultsWrapper, pd.
        ↪ DataFrame]: Fitted OLS model and design matrix after dropping insignificant
        ↪ lag-2 terms.

    Raises:
        None
    """
    drop_cols = [
        c for c in X.columns if c.endswith("_lag2") and c in model.pvalues.
        ↪ index and model.pvalues[c] > alpha
    ]
    if drop_cols:
        X2 = X.drop(columns=drop_cols)
        # Align X2 and y to the same index
        X2, y_aligned = X2.align(y, join="inner", axis=0)
        model2 = sm.OLS(y_aligned, X2).fit()
        return model2, X2
    return model, X

```

1.6.3 SVAR Fitting Function

The recursively fit SVAR function is done using `statsmodels.api.OLS` (Ordinary Least Squares) as specified in the paper, in order to create the linear environment. The linear economy is represented using $\hat{f}^m = C^m + \alpha^{m'} s_t^m$ where $\hat{f}^m$ represents a linear function comprised of the collected inputs from the s_t^m vector. C^m and α^m for $m \in \{pi, y\}$ are the constant and vector coefficients. If we expand s_t^m for $m \in \{\pi, y\}$ we get the input vectors: $s_t^y = (y_{t-1}, \pi_{t-1}, i_{t-1}, i_{t-2})$ and $s_t^\pi = (y_t, y_{t-1}, y_{t-2}, \pi_{t-1}, \pi_{t-2}, i_{t-1})$. It is important to note that the only vector which relies on a contemporaneous term is s_t^π which contains contemporaneous y_t . If we substitute our input vectors for the linear environment we arrive at the linear system as follows: $y_t = C^y + \alpha_{y,1}^y y_{t-1} + \alpha_{\pi,1}^y \pi_{t-1} + \alpha_{i,1}^y i_{t-1} + \alpha_{i,2}^y i_{t-2} + \epsilon_t^y$ and $\pi_t = C^\pi + \alpha_{y,0}^\pi y_t + \alpha_{y,1}^\pi y_{t-1} + \alpha_{y,2}^\pi y_{t-2} + \alpha_{\pi,1}^\pi \pi_{t-1} + \alpha_{\pi,2}^\pi \pi_{t-2} + \alpha_{i,1}^\pi i_{t-1} + \epsilon_t^\pi$ which can be visualised as:

$$\hat{f}^m = C^m + \alpha^{m'} s_t^m, m \in \{\pi, y\} =$$

$$\begin{cases} C^y + \alpha_{y,1}^y y_{t-1} + \alpha_{\pi,1}^y \pi_{t-1} + \alpha_{i,1}^y i_{t-1} + \alpha_{i,2}^y i_{t-2} + \epsilon_t^y & m = y \\ C^\pi + \alpha_{y,0}^\pi y_t + \alpha_{y,1}^\pi y_{t-1} + \alpha_{y,2}^\pi y_{t-2} + \alpha_{\pi,1}^\pi \pi_{t-1} + \alpha_{\pi,2}^\pi \pi_{t-2} + \alpha_{i,1}^\pi i_{t-1} + \epsilon_t^\pi & m = \pi \end{cases}$$

```
[42]: def fit_linear_svar(df: pd.DataFrame, alpha_drop: float = 0.10) -> SVARResults:
    """
    Fit the paper's restricted linear SVAR environment via recursive OLS.

    Args:
        df (pd.DataFrame): Quarterly data with a (PeriodIndex or DatetimeIndex)
        ↪ sorted in time and columns ['pi', 'y', 'i'] for inflation (), output gap
        ↪ (y), and the federal funds rate (i).
        alpha_drop (float): Significance level for pruning lag-2 terms.

    Returns:
        SVARResults: Dataclass containing fitted models, design matrices, and
        ↪ residuals.

    Raises:
        AssertionError: If the required columns are missing.
    """
    # Check input
    assert {"pi", "y", "i"}.issubset(df.columns), "df must have columns: 'pi',
    ↪ 'y', 'i'"

    # Work copy with lags up to 2
    work = df.copy().sort_index()
    for c in ("pi", "y", "i"):
        work[f"{c}_lag1"] = work[c].shift(1)
        work[f"{c}_lag2"] = work[c].shift(2)

    # Output gap equation: y_t ~ y_{t-1}, pi_{t-1}, i_{t-1}, i_{t-2}
    y_target = work["y"]
    X_y = work[["y_lag1", "pi_lag1", "i_lag1", "i_lag2"]]
    X_y = sm.add_constant(X_y, has_constant="add")
    ymask = X_y.notna().all(axis=1) & y_target.notna()
    mod_y = sm.OLS(y_target[ymask], X_y[ymask]).fit()
    mod_y, X_y = drop_insignificant_lag2(mod_y, X_y[ymask], y_target,
    ↪ alpha_drop)

    # Inflation equation: pi_t ~ y_t, y_{t-1}, y_{t-2}, pi_{t-1}, pi_{t-2},
    ↪ i_{t-1}
    pi_target = work["pi"]
    X_pi = work[["y", "y_lag1", "y_lag2", "pi_lag1", "pi_lag2", "i_lag1"]]
    X_pi = sm.add_constant(X_pi, has_constant="add")
    pimask = X_pi.notna().all(axis=1) & pi_target.notna()
    mod_pi = sm.OLS(pi_target[pimask], X_pi[pimask]).fit()
```

```

mod_pi, X_pi = drop_insignificant_lag2(mod_pi, X_pi[pimask], pi_target,
↪alpha_drop)

# Residuals (aligned to design matrices)
resid_y = y_target.loc[X_y.index] - mod_y.predict(X_y)
resid_y.name = "eps_y"
resid_pi = pi_target.loc[X_pi.index] - mod_pi.predict(X_pi)
resid_pi.name = "eps_pi"

return SVARResults(
    mod_y=mod_y,
    mod_pi=mod_pi,
    X_y=X_y,
    X_pi=X_pi,
    resid_y=resid_y,
    resid_pi=resid_pi,
)

```

1.6.4 SVAR Series Equations:

Output gap equation:

$$y_t = C^y + \alpha_{y,1}^y y_{t-1} + \alpha_{\pi,1}^y \pi_{t-1} + \alpha_{i,1}^y i_{t-1} + \alpha_{i,2}^y i_{t-2} + \varepsilon_t^y$$

Inflation equation (note the contemporaneous y_t term):

$$\pi_t = C^\pi + \alpha_{y,0}^\pi y_t + \alpha_{y,1}^\pi y_{t-1} + \alpha_{y,2}^\pi y_{t-2} + \alpha_{\pi,1}^\pi \pi_{t-1} + \alpha_{\pi,2}^\pi \pi_{t-2} + \alpha_{i,1}^\pi i_{t-1} + \varepsilon_t^\pi$$

```

[43]: # Fit SVAR
svar = fit_linear_svar(df)

print("\n=== Output gap equation ===")
print(svar.mod_y.summary())
print("\n=== Inflation equation ===")
print(svar.mod_pi.summary())

# Sanity Checks:
print("\nResidual std (y, pi):", svar.resid_y.std().round(3), svar.resid_pi.
↪std().round(3))
print("Samples used (y, pi):", svar.mod_y.nobs, svar.mod_pi.nobs)

```

=== Output gap equation ===

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:          0.889
Model:                OLS      Adj. R-squared:      0.883

```

```

Method:                Least Squares    F-statistic:                146.0
Date:                  Thu, 09 Oct 2025  Prob (F-statistic):        4.99e-34
Time:                  12:43:25          Log-Likelihood:            -50.085
No. Observations:      78              AIC:                      110.2
Df Residuals:          73              BIC:                      122.0
Df Model:              4
Covariance Type:       nonrobust

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.3473	0.181	1.924	0.058	-0.012	0.707
y_lag1	0.9318	0.058	15.982	0.000	0.816	1.048
pi_lag1	-0.0595	0.086	-0.690	0.492	-0.231	0.112
i_lag1	0.2003	0.154	1.299	0.198	-0.107	0.508
i_lag2	-0.2385	0.138	-1.732	0.087	-0.513	0.036
Omnibus:	2.870	Durbin-Watson:	1.822			
Prob(Omnibus):	0.238	Jarque-Bera (JB):	2.367			
Skew:	-0.423	Prob(JB):	0.306			
Kurtosis:	3.116	Cond. No.	31.1			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

=== Inflation equation ===

OLS Regression Results

```

=====
Dep. Variable:          pi    R-squared:                0.951
Model:                  OLS    Adj. R-squared:            0.947
Method:                 Least Squares    F-statistic:                230.2
Date:                  Thu, 09 Oct 2025  Prob (F-statistic):        1.82e-44
Time:                  12:43:25          Log-Likelihood:            24.128
No. Observations:      78              AIC:                      -34.26
Df Residuals:          71              BIC:                      -17.76
Df Model:              6
Covariance Type:       nonrobust

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.1178	0.074	1.588	0.117	-0.030	0.266
y	-0.0712	0.045	-1.570	0.121	-0.162	0.019
y_lag1	0.2092	0.066	3.154	0.002	0.077	0.341
y_lag2	-0.1110	0.047	-2.361	0.021	-0.205	-0.017
pi_lag1	1.3027	0.106	12.313	0.000	1.092	1.514
pi_lag2	-0.3245	0.110	-2.941	0.004	-0.545	-0.104
i_lag1	-0.0146	0.015	-0.949	0.346	-0.045	0.016

Omnibus:	1.161	Durbin-Watson:	2.103
Prob(Omnibus):	0.560	Jarque-Bera (JB):	0.943
Skew:	-0.269	Prob(JB):	0.624
Kurtosis:	2.975	Cond. No.	46.0

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Residual std (y, pi): 0.463 0.179

Samples used (y, pi): 78.0 78.0

1.6.5 Table 9: Estimation Results of Restricted SVAR (Comparison)

```
[44]: from statsmodels.stats.stattools import durbin_watson
from statsmodels.stats.diagnostic import acorr_breusch_godfrey
from IPython.display import Markdown, display

table_nine_str = fr"""
| Parameters | Bundesbank Estimate | p-Value | Local OLS_
| Estimate | p-Value | | |
|---|---|---|---|
| **Output gap equation** | | | |
| $C^y$ | 0.3834 | 0.0351 | {svar.mod_y.
| params['const']:.4f} | {svar.mod_y.pvalues['const']:.4f} |
| $a^y_{\{y,1\}}$ | 0.9084 | 0.0000 | {svar.mod_y.
| params['y_lag1']:.4f} | {svar.mod_y.pvalues['y_lag1']:.4f} |
| $a^y_{\{\pi,1\}}$ | -0.1437 | 0.1409 | {svar.mod_y.
| params['pi_lag1']:.4f} | {svar.mod_y.pvalues['pi_lag1']:.4f} |
| $a^y_{\{i,1\}}$ | 0.2726 | 0.0661 | {svar.mod_y.
| params['i_lag1']:.4f} | {svar.mod_y.pvalues['i_lag1']:.4f} |
| $a^y_{\{i,2\}}$ | -0.2896 | 0.0313 | {svar.mod_y.
| params['i_lag2']:.4f} | {svar.mod_y.pvalues['i_lag2']:.4f} |
| $\bar{\{R\}}^2$ | 0.9100 | | {svar.mod_y.
| rsquared:.4f} | |
|-----|-----|-----|-----|

```

MSE	0.2108		{svar.mod_y.
↪mse_resid:.4f}			↪
↪			
$\hat{\sigma}_{\{\varepsilon_1\}}$	0.2136		{svar.mod_y.
↪scale ** 0.5:.4f}			↪
↪			
DW	1.8206		↪
↪{durbin_watson(svar.mod_y.resid):.4f}			↪
↪			
LM(1)	3.1037	0.1644	↪
↪{acorr_breusch_godfrey(svar.mod_y, 1)[2]:.4f}		{acorr_breusch_godfrey(svar.	
↪mod_y, 1)[3]:.4f}			↪
Inflation equation			↪
↪			↪
↪			
C^{π}	0.1035	0.1659	{svar.mod_pi.
↪params['const']:.4f}		{svar.mod_pi.pvalues['const']:.4f}	↪
↪			
$a^{\pi}_{\{y,0\}}$	-0.0655	0.1578	{svar.mod_pi.
↪params['y']:.4f}		{svar.mod_pi.pvalues['y']:.4f}	↪
↪			
$a^{\pi}_{\{y,1\}}$	0.1970	0.0048	{svar.mod_pi.
↪params['y_lag1']:.4f}		{svar.mod_pi.pvalues['y_lag1']:.4f}	↪
↪			
$a^{\pi}_{\{y,2\}}$	-0.1121	0.0163	{svar.mod_pi.
↪params['y_lag2']:.4f}		{svar.mod_pi.pvalues['y_lag2']:.4f}	↪
↪			
$a^{\pi}_{\{\pi,1\}}$	1.2970	0.0000	{svar.mod_pi.
↪params['pi_lag1']:.4f}		{svar.mod_pi.pvalues['pi_lag1']:.4f}	↪
↪			
$a^{\pi}_{\{\pi,2\}}$	-0.3116	0.0076	{svar.mod_pi.
↪params['pi_lag2']:.4f}		{svar.mod_pi.pvalues['pi_lag2']:.4f}	↪
↪			
$a^{\pi}_{\{i,1\}}$	-0.0122	0.4174	{svar.mod_pi.
↪params['i_lag1']:.4f}		{svar.mod_pi.pvalues['i_lag1']:.4f}	↪
↪			
$\bar{R}^2$	0.9450		{svar.mod_pi.
↪rsquared:.4f}			↪
↪			
MSE	0.0326		{svar.mod_pi.
↪mse_resid:.4f}			↪
↪			
$\hat{\sigma}_{\{\varepsilon_2\}}$	0.0330		{svar.mod_pi.
↪scale ** 0.5:.4f}			↪
↪			

```

| DW | 2.1095 | |
↪{durbin_watson(svar.mod_pi.resid):.4f} |
↪
| LM(1) | 2.5542 | 0.0696 |
↪{acorr_breusch_godfrey(svar.mod_pi, 1)[2]:.4f} | {acorr_breusch_godfrey(svar.
↪mod_pi, 1)[3]:.4f} |

\* Residual std dev from OLS output.

display(Markdown(table_nine_str))

```

Parameters	Bundesbank Estimate	p- Value	Local OLS Estimate	p-Value
Output gap equation				
C^y	0.3834	0.0351	0.3473	0.0582
$a_{y,1}^y$	0.9084	0.0000	0.9318	0.0000
$a_{\pi,1}^y$	-0.1437	0.1409	-0.0595	0.4922
$a_{i,1}^y$	0.2726	0.0661	0.2003	0.1980
$a_{i,2}^y$	-0.2896	0.0313	-0.2385	0.0875
$\bar{R}^2$	0.9100		0.8889	
MSE	0.2108		0.2260	
$\hat{\sigma}_{\varepsilon_1}$	0.2136		0.4754	
DW	1.8206		1.8220	
LM(1)	3.1037	0.1644	0.6285	0.4305
Inflation equation				
C^π	0.1035	0.1659	0.1178	0.1166
$a_{y,0}^\pi$	-0.0655	0.1578	-0.0712	0.1209
$a_{y,1}^\pi$	0.1970	0.0048	0.2092	0.0024
$a_{y,2}^\pi$	-0.1121	0.0163	-0.1110	0.0210
$a_{\pi,1}^\pi$	1.2970	0.0000	1.3027	0.0000
$a_{\pi,2}^\pi$	-0.3116	0.0076	-0.3245	0.0044
$a_{i,1}^\pi$	-0.0122	0.4174	-0.0146	0.3460
$\bar{R}^2$	0.9450		0.9511	
MSE	0.0326		0.0346	
$\hat{\sigma}_{\varepsilon_2}$	0.0330		0.1861	
DW	2.1095		2.1028	
LM(1)	2.5542	0.0696	1.5639	0.2153

* Residual std dev from OLS output.

1.6.6 Complete Equations with Substituted Coefficients

```
[45]: y_t_eq_str = fr"""
$y_t = {svar.mod_y.params['const']:.4f} + ({svar.mod_y.params['y_lag1']:.
↪4f})y_{t-1} + ({svar.mod_y.params['pi_lag1']:.4f}){\pi}_{t-1} + ({svar.
↪mod_y.params['i_lag1']:.4f})i_{t-1} + ({svar.mod_y.params['i_lag2']:.
↪4f})i_{t-2} + {\epsilon}_t^y$
"""

pi_t_eq_str = fr"""
${\pi}_t = {svar.mod_pi.params['const']:.4f} + ({svar.mod_pi.params['y']:.
↪4f})y_t + ({svar.mod_pi.params['y_lag1']:.4f})y_{t-1} + ({svar.mod_pi.
↪params['y_lag2']:.4f})y_{t-2} + ({svar.mod_pi.params['pi_lag1']:.
↪4f}){\pi}_{t-1} + ({svar.mod_pi.params['pi_lag2']:.4f}){\pi}_{t-2} +
↪({svar.mod_pi.params['i_lag1']:.4f})i_{t-1} + {\epsilon}_t^{\pi}$
"""

display(Markdown(
    r"""The new equations with their coefficients are as follows: """))
display(Markdown(y_t_eq_str))
display(Markdown(pi_t_eq_str))
display(Markdown(
    r"""Note: None of the lag 2 terms were dropped as they all had significant
↪p-values relative to our o-drop parameter. """)))
```

The new equations with their coefficients are as follows:

$$y_t = 0.3473 + (0.9318)y_{t-1} + (-0.0595)\pi_{t-1} + (0.2003)i_{t-1} + (-0.2385)i_{t-2} + \epsilon_t^y$$

$$\pi_t = 0.1178 + (-0.0712)y_t + (0.2092)y_{t-1} + (-0.1110)y_{t-2} + (1.3027)\pi_{t-1} + (-0.3245)\pi_{t-2} + (-0.0146)i_{t-1} + \epsilon_t^\pi$$

Note: None of the lag 2 terms were dropped as they all had significant p-values relative to our o-drop parameter.

1.7 Plot Findings from Sample Data Against Observed Data

1.7.1 Static Inputs for Out of Sample Fitting

```
[46]: import datetime

# Forecasting data Time Period: Q3 2007 - Now
os_start_date = "2007-07-01"
os_end_date = datetime.date.today().strftime("%Y-%m-%d")
```

1.7.2 Fetch Comparison Data and Construct Series

```
[47]: # Fetch data from FRED
os_gdpdef_yoy = fred.get_series_observations(
    series_id="GDPDEF",
    observation_start=os_start_date,
    observation_end=os_end_date,
    frequency=freq,
    units="pc1"
)

os_real_gdp = fred.get_series_observations(
    series_id="GDPPI",
    observation_start=os_start_date,
    observation_end=os_end_date,
    frequency=freq,
    units="lin"
)

os_potential_gdp = fred.get_series_observations(
    series_id="GDPPI",
    observation_start=os_start_date,
    observation_end=os_end_date,
    frequency=freq,
    units="lin"
)

os_effective_ffr = fred.get_series_observations(
    series_id="FEDFUNDS",
    observation_start=os_start_date,
    observation_end=os_end_date,
    frequency=freq,
    aggregation_method="avg"
)

# Convert to Series with proper names
os_pi = to_series(os_gdpdef_yoy, "os_p")
os_y = to_series(os_real_gdp, "os_y")
os_y_pot = to_series(os_potential_gdp, "os_y_pot")
os_i = to_series(os_effective_ffr, "i")

# Create y_gap series (Output Gap from Log Difference)
os_y_gap = 100.0 * (os_y / os_y_pot - 1.0)
os_y_gap.name = "os_y_gap"

# Check calculated series
```

```

print(os_pi.head())
print(os_pi.tail())
print(os_y_gap.head())
print(os_y_gap.tail())
print(os_i.head())
print(os_i.tail())

```

date

2007-07-01	2.57376
2007-10-01	2.62239
2008-01-01	2.03211
2008-04-01	1.81848
2008-07-01	2.02788

Name: os_p, dtype: float64

date

2024-04-01	2.66322
2024-07-01	2.26337
2024-10-01	2.46749
2025-01-01	2.57530
2025-04-01	2.46151

Name: os_p, dtype: float64

date

2007-07-01	1.413096
2007-10-01	1.542354
2008-01-01	0.604280
2008-04-01	0.709002
2008-07-01	-0.292576

Name: os_y_gap, dtype: float64

date

2024-10-01	2.047311
2025-01-01	1.312168
2025-04-01	1.689360
2025-07-01	NaN
2025-10-01	NaN

Name: os_y_gap, dtype: float64

date

2007-07-01	5.07
2007-10-01	4.50
2008-01-01	3.18
2008-04-01	2.09
2008-07-01	1.94

Name: i, dtype: float64

date

2024-07-01	5.26
2024-10-01	4.65
2025-01-01	4.33
2025-04-01	4.33
2025-07-01	4.29

Name: i, dtype: float64

```
[48]: # Merge into one OOS DataFrame
df_oos = pd.concat([os_pi.rename("pi"), os_y_gap.rename("y"), os_i], axis=1).
    ↪ dropna().sort_index()
df_oos.index = df_oos.index.to_period("Q")

print(df_oos.head(), "\n")
print("OOS span:", df_oos.index.min(), "→", df_oos.index.max(), "| rows:",
    ↪ len(df_oos))
```

	pi	y	i
date			
2007Q3	2.57376	1.413096	5.07
2007Q4	2.62239	1.542354	4.50
2008Q1	2.03211	0.604280	3.18
2008Q2	1.81848	0.709002	2.09
2008Q3	2.02788	-0.292576	1.94

OOS span: 2007Q3 → 2025Q2 | rows: 72

1.7.3 Roll Values Forward using Forecast Coefficients

```
[49]: # Set df_train reference
df_train = df.copy()
```

Check SVAR results summary var assignment before use.

```
[50]: # Coefficients from your regression output
coef_y = svar.mod_y.params.to_dict()
coef_pi = svar.mod_pi.params.to_dict()

# Iterative Print Loop (for output readability)
for key in coef_y.keys():
    print(f"{key}: {coef_y[key]:.4f}")

for key in coef_pi.keys():
    print(f"{key}: {coef_pi[key]:.4f}")
```

const: 0.3473
y_lag1: 0.9318
pi_lag1: -0.0595
i_lag1: 0.2003
i_lag2: -0.2385
const: 0.1178
y: -0.0712
y_lag1: 0.2092
y_lag2: -0.1110
pi_lag1: 1.3027

```
pi_lag2: -0.3245
i_lag1: -0.0146
```

```
[51]: # Pull coefficients directly from your fitted models
coef_y = svar.mod_y.params.to_dict()
coef_pi = svar.mod_pi.params.to_dict()
# print(coef_y, coef_pi) # sanity: keys should include
#   ↳ 'const', 'y_lag1', 'pi_lag1', 'i_lag1', 'i_lag2', and for pi:
#   ↳ 'const', 'y', 'y_lag1', 'y_lag2', 'pi_lag1', 'pi_lag2', 'i_lag1'

df_all = pd.concat([df_train, df_oos]).sort_index()
df_all["y_hat"] = np.nan
df_all["pi_hat"] = np.nan

def lag_hat_or_actual(t, col, k):
    """Use forecast if available, else actual, at lag k."""
    prev = df_all.at[t - k, f"{col}_hat"]
    return prev if pd.notna(prev) else df_all.at[t - k, col]

# start at a point where t-2 exists
start = max(df_all.index.min() + 2, pd.Period("2007Q3", "Q"))

for t in df_all.loc[start:].index:
    # --- Output gap: y_t ---
    y_lag1 = lag_hat_or_actual(t, "y", 1)
    pi_lag1 = lag_hat_or_actual(t, "pi", 1)
    i_lag1 = df_all.at[t - 1, "i"]
    i_lag2 = df_all.at[t - 2, "i"]

    df_all.at[t, "y_hat"] = (
        coef_y["const"]
        + coef_y["y_lag1"] * y_lag1
        + coef_y["pi_lag1"] * pi_lag1
        + coef_y["i_lag1"] * i_lag1
        + coef_y["i_lag2"] * i_lag2
    )

    # --- Inflation: pi_t (uses contemporaneous y_hat) ---
    y_t = df_all.at[t, "y_hat"]
    y_lag1 = lag_hat_or_actual(t, "y", 1)
    y_lag2 = lag_hat_or_actual(t, "y", 2)
    pi_lag1 = lag_hat_or_actual(t, "pi", 1)
    pi_lag2 = lag_hat_or_actual(t, "pi", 2)
    i_lag1 = df_all.at[t - 1, "i"]

    df_all.at[t, "pi_hat"] = (
        coef_pi["const"]
```

```

+ coef_pi["y"]      * y_t
+ coef_pi["y_lag1"] * y_lag1
+ coef_pi["y_lag2"] * y_lag2
+ coef_pi["pi_lag1"] * pi_lag1
+ coef_pi["pi_lag2"] * pi_lag2
+ coef_pi["i_lag1"] * i_lag1
)

```

1.7.4 Plot Forecast vs. Actual

```

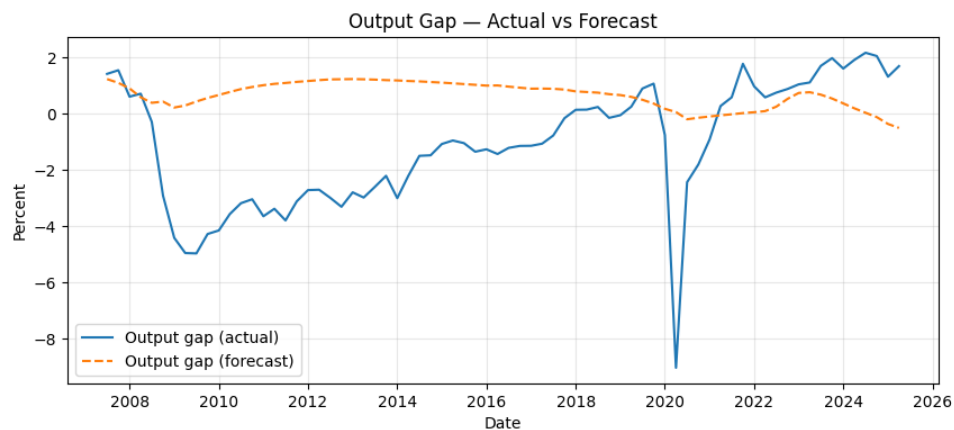
[52]: import matplotlib.pyplot as plt

sub = df_all.loc["2007Q3":]

# Output gap
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(sub.index.to_timestamp(), sub["y"], label="Output gap (actual)")
ax.plot(sub.index.to_timestamp(), sub["y_hat"], "--", label="Output gap (forecast)")
ax.set_title("Output Gap - Actual vs Forecast")
ax.set_xlabel("Date")
ax.set_ylabel("Percent")
ax.legend()
ax.grid(True, alpha=0.3)
plt.show()

# Save figures for later use
figure_five = fig

```



```

[53]: # Inflation
fig, ax = plt.subplots(figsize=(10, 4))

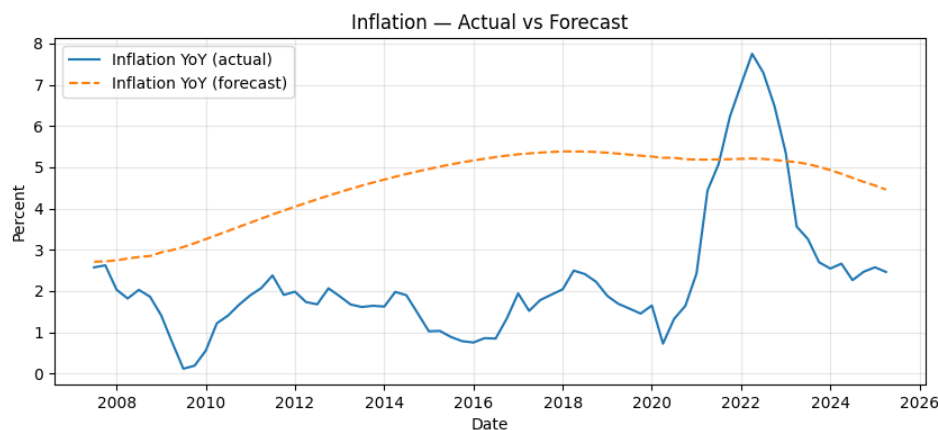
```

```

ax.plot(sub.index.to_timestamp(), sub["pi"], label="Inflation YoY (actual)")
ax.plot(sub.index.to_timestamp(), sub["pi_hat"], "--", label="Inflation YoY_
↳(forecast)")
ax.set_title("Inflation - Actual vs Forecast")
ax.set_xlabel("Date")
ax.set_ylabel("Percent")
ax.legend()
ax.grid(True, alpha=0.3)
plt.show()

# Save figures for later use
figure_six = fig

```



1.8 Additional SVAR Output (In Sample Data Range)

```

[54]: # Training window (paper window)
df_train = df.loc["1987Q3":"2007Q2"].copy()

# Predict on the exact in-sample design matrices
y_fit = svar.mod_y.predict(svar.X_y)
pi_fit = svar.mod_pi.predict(svar.X_pi)

# Sanity: indexes should match design matrices you trained on
assert y_fit.index.equals(svar.X_y.index)
assert pi_fit.index.equals(svar.X_pi.index)

# Build aligned frame on the training index
fit_df = pd.DataFrame(index=df_train.index)
fit_df["y_actual"] = df_train["y"]
fit_df["pi_actual"] = df_train["pi"]
fit_df["y_fit"] = y_fit.reindex(fit_df.index)

```

```

fit_df["pi_fit"]    = pi_fit.reindex(fit_df.index)

# Residuals (in-sample)
resid_y = (fit_df["y_actual"] - fit_df["y_fit"])
resid_pi = (fit_df["pi_actual"] - fit_df["pi_fit"])

# Squared errors
fit_df["se_y_svar"] = resid_y**2
fit_df["se_pi_svar"] = resid_pi**2

# Drop rows without a fitted value (initial lags)
fit_df = fit_df.dropna(subset=["y_fit", "pi_fit"])

# Quick diagnostics
print(fit_df[["se_y_svar", "se_pi_svar"]].head())
print("In-sample std(resid):",
      round(resid_y.dropna().std(), 3),
      round(resid_pi.dropna().std(), 3))

```

	se_y_svar	se_pi_svar
date		
1988Q1	0.025177	0.000604
1988Q2	0.486089	0.119382
1988Q3	0.017772	0.103834
1988Q4	0.337998	0.002397
1989Q1	0.168499	0.067581
In-sample std(resid):	0.463	0.179

1.8.1 Figure 4 Output Gap Fit: Squared Errors (Replication)

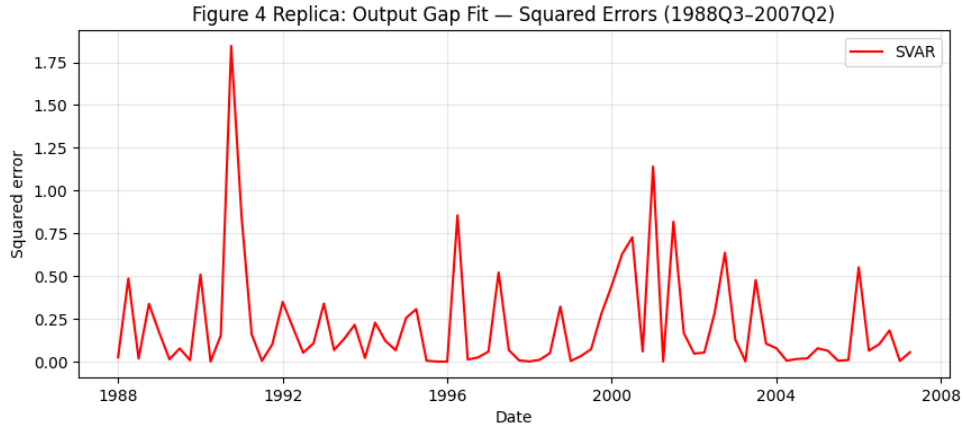
```

[55]: import matplotlib.pyplot as plt

# Figure 4: Output gap fit - squared errors
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(fit_df.index.to_timestamp(), fit_df["se_y_svar"], color="red",
        label="SVAR", linewidth=1.5)
ax.set_title("Figure 4 Replica: Output Gap Fit - Squared Errors (1988Q3-2007Q2)")
ax.set_xlabel("Date"); ax.set_ylabel("Squared error"); ax.grid(True, alpha=0.3)
ax.legend()
plt.show()

# Save figures for later use
figure_seven = fig

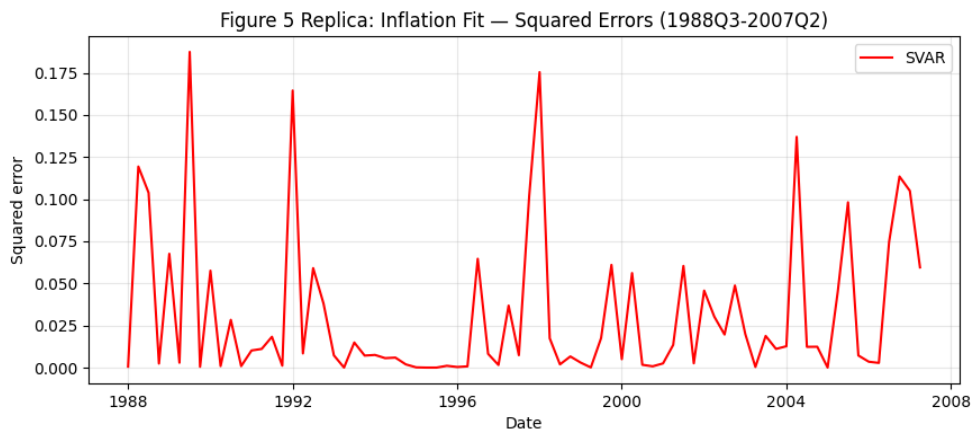
```



1.8.2 Figure 5 Inflation Fit: Squared Errors (Replication)

```
[56]: # Figure 5: Inflation fit - squared errors
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(fit_df.index.to_timestamp(), fit_df["se_pi_svar"], color="red",
        label="SVAR", linewidth=1.5)
ax.set_title("Figure 5 Replica: Inflation Fit - Squared Errors (1988Q3-2007Q2)")
ax.set_xlabel("Date"); ax.set_ylabel("Squared error"); ax.grid(True, alpha=0.3)
ax.legend()
plt.show()

# Save figures for later use
figure_eight = fig
```



Note: The y-axis scale is different from the paper due to our model using the correct units (YoY percent, not decimals). In addition The smaller squared errors reflect a tighter fit in the replication

2 Citations

1. The pandas development team. pandas-dev/pandas: Pandas [Computer software]. <https://doi.org/10.5281/zenodo.3509134>
2. Sunder, N. (2025). fedfred: A Python client for the Federal Reserve Economic Database (FRED) API (Version 2.1.5) [Computer software]. <https://doi.org/10.5281/zenodo.17180397>
3. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
4. Skipper, S., & Josef, P. (2010). statsmodels: Econometric and statistical modeling with python. 9th Python in Science Conference
5. Python Software Foundation. (2025). Python (Version 3.13.7) [Computer software]. Python Software Foundation. <https://www.python.org/>
6. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
7. Hinterlang, N., & Tänzer, A. (2021). Optimal monetary policy using reinforcement learning. Deutsche Bundesbank. <https://books.google.com/books?id=yjjIzgEACAAJ>